Capítulo 1

Introducción a las Redes de Computadoras

Parte 3

Metas

- **Vamos a estudiar**:
  1. Nivel físico
  2. Conexión directa entre máquinas
  3. Conectando redes de área local
  4. Conectando redes de área amplia
  5. Redes proveedoras de contenido
  6. **Conectando procesos**
  7. Aplicaciones de red
  8. Modelo de capas de una red

Bloque 6: Conectar Procesos

- Ya sabemos cómo conectar máquinas a escala global; ahora debemos lograr que los programas dentro de esas máquinas puedan comunicarse.
- Por esto los mensajes deben indicar cuál es el programa receptor, introduciendo la idea de puertos.
- Como la red subyacente a una aplicación no se ocupa de manejar todas sus imperfecciones, se requiere cubrir asuntos como ordenamiento de paquetes, control de congestión, control de flujo y comunicación confiable.
- Este bloque responde a: **¿cómo logramos que programas específicos en máquinas distintas se comuniquen correctamente, incluso cuando la red es imperfecta?**

Bloque 6: Conectar Procesos

- **¿Con lo que vimos hasta aquí, si dos máquinas tienen varios programas, puedo hacer que un programa de una máquina se comunique con otro programa de la otra máquina (entre ellas hay una inter red)?**

Bloque 6: Conectar Procesos

- **¿ Con lo que vimos hasta aquí, si dos máquinas tienen varios programas, puedo hacer que un programa de una máquina se comunique con otro programa de la otra máquina (entre ellas hay una inter red)?**
- No porque solo vimos envío de mensajes entre máquinas ya sea dentro de una red o en una inter red.

Bloque 6: Conectar Procesos

- **¿Cómo distinguir los mensajes para un programa con los mensajes para el otro programa?**

Bloque 6: Conectar Procesos

- **¿Cómo distinguir los mensajes para un programa con los mensajes para el otro programa?**
- Los mensajes deben indicar cual es el programa receptor.
- **¿Cómo se puede indicar esto?**

Bloque 6: Conectar Procesos

- **¿Cómo distinguir los mensajes para un programa con los mensajes para el otro programa?**
- Los mensajes deben indicar cual es el programa receptor.
- **¿Cómo se puede indicar esto?**
- Usar un identificador de programa (en internet se lo llama puerto)

Bloque 6: Conectar Procesos

- Ahora nos concentraremos en responder la siguiente pregunta.
- **¿Qué problemas tienen las redes que conectan LANs que no resolvimos?**
- Para responderla hay que analizar varios aspectos del funcionamiento de las redes.

Bloque 6: Conectar Procesos
Ordenamiento de paquetes

- **¿Si mandamos dos paquetes de un mismo mensaje, esos mensajes van a seguir la misma ruta?**

Bloque 6: Conectar Procesos
Ordenamiento de paquetes

- **¿Si mandamos dos paquetes de un mismo mensaje, esos paquetes van a seguir la misma ruta?**
- No necesariamente, puede que cambien las rutas entre paquetes porque se ejecutó el algoritmo de enrutamiento.
- **Si los dos paquetes de un mensaje son enviados, van a llegar en orden?** (**ayuda**: pensar en diferentes cosas que les pueden pasar a los paquetes)

Bloque 6: Conectar Procesos
Ordenamiento de paquetes

- **¿Si mandamos dos paquetes de un mismo mensaje, esos paquetes van a seguir la misma ruta?**
- No necesariamente, puede que cambien las rutas entre paquetes porque se ejecutó el algoritmo de enrutamiento.
- **Si los dos paquetes de un mensaje son enviados, van a llegar en orden?**
- No necesariamente.
  - El segundo paquete sigue una ruta mas corta y llega antes (por cambio de rutas)
  - El primer paquete se pierde y debe ser reenviado; lo que causa que el segundo paquete llegue antes.
- **Conclusión**: los paquetes de un mismo mensaje pueden llegar fuera de orden.
- Entonces es necesario ordenar los paquetes de un mensaje porque una aplicación de red necesita procesar el mensaje completo ordenado.

Bloque 6: Conectar Procesos
Control de congestión

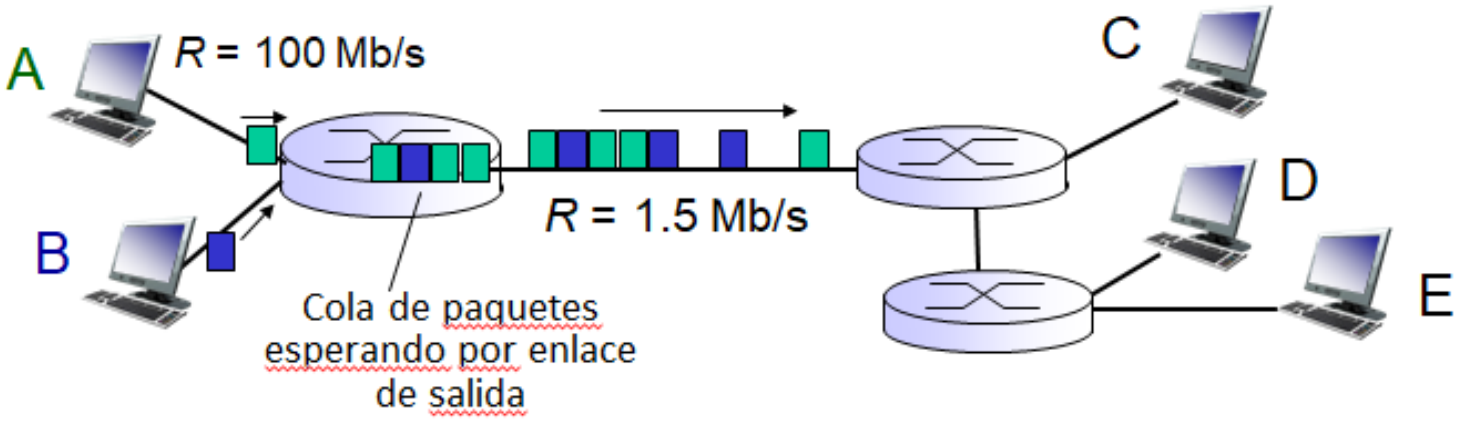
- Resolver el ordenamiento garantiza que un flujo pueda reconstruirse correctamente.
  - Pero este mecanismo asume que la red puede transportar esos paquetes sin saturarse.
- Hasta ahora no dijimos nada sobre qué ocurre cuando muchos flujos simultáneos compiten por la misma red.
  - En ese escenario, aunque cada flujo pueda ordenar sus paquetes, la red puede saturarse, perder paquetes masivamente o volverse inestable.
  - A este fenómeno se lo llama **congestión** y requiere mecanismos específicos para evitar que la red colapse cuando múltiples procesos intentan comunicarse al mismo tiempo.

Bloque 6: Conectar Procesos
Control de congestión

- **¿Qué pasa si en la línea de salida de un enrutador llegan demasiados paquetes a reenviar?** (**ayuda**: recordar que hay una cola en la línea de salida, y que la línea de salida tiene una cierta capacidad de almacenamiento)

Bloque 6: Conectar Procesos
Control de congestión

- **¿Qué pasa si en la línea de salida de un enrutador llegan demasiados paquetes a reenviar?**
  - La cola de salida se hace muy grande y el reenvío se pone lento.
  - La capacidad de la línea de salida se puede saturar y entonces comienzan a perderse paquetes.
- A este fenómeno se lo llama **congestión**.
- La congestión ocurre cuando la red no puede manejar la carga de paquetes que recibe de manera aceptable



Bloque 6: Conectar Procesos
Control de congestión

- **¿Cómo controlar la congestión (dar alguna idea)?**

Bloque 6: Conectar Procesos
Control de congestión

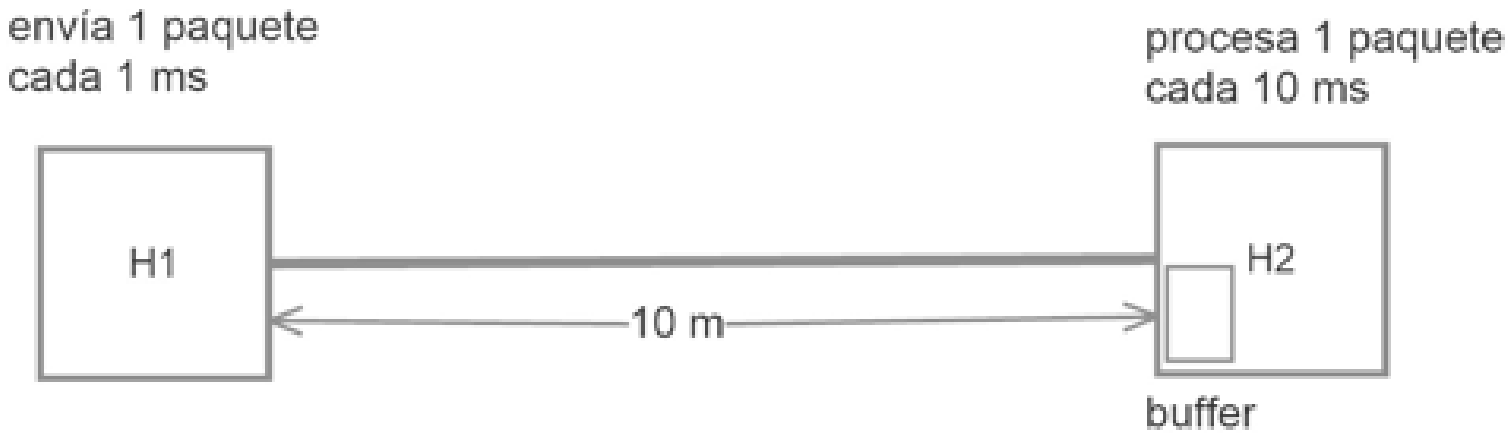
- **¿Cómo controlar la congestión (dar alguna idea)?**
- Las computadoras emisoras se enteran de la congestión y reducen el tráfico de salida.
- Esta idea puede ser implementada de muchas maneras como veremos más adelante.
- A esto se le llama **control de congestión**.

Bloque 6: Conectar Procesos
Control de Flujo

- El control de congestión evita que la red se sature cuando muchos flujos compiten por los mismos enrutadores.
  - Pero este mecanismo solo considera el estado de la red, no el estado de los procesos que se comunican.
  - Incluso si no hay congestión, un receptor puede saturarse si el emisor envía datos más rápido de lo que puede procesarlos.
  - Cuando eso ocurre, el receptor pierde paquetes y el emisor probablemente va a interpretar esas pérdidas como congestión, reaccionando de manera incorrecta.
  - Para evitar que el emisor sobrecargue al receptor, es necesario definir un mecanismo adicional de **control de flujo**.

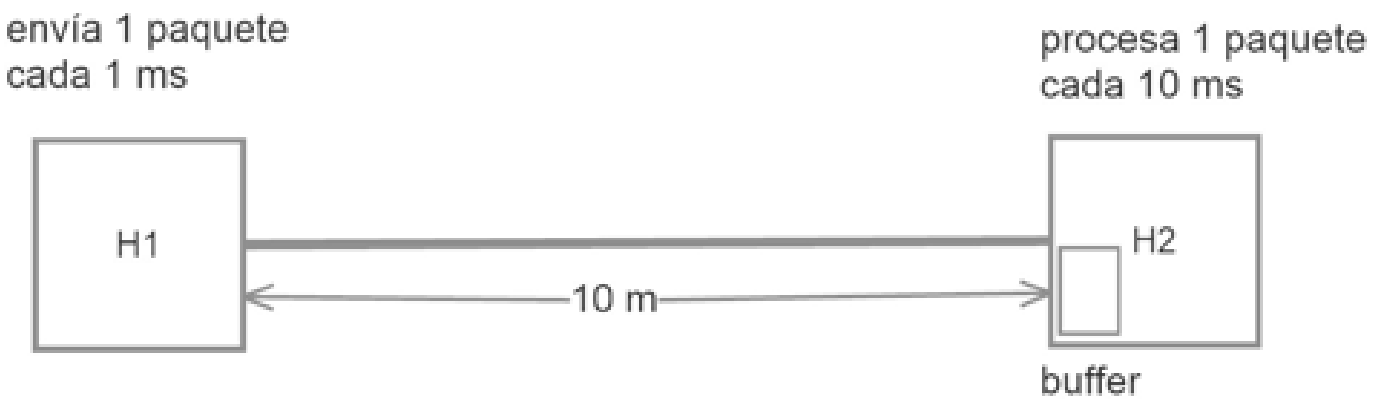
Bloque 6: Conectar Procesos
Control de Flujo

- **¿Qué pasa si un emisor envía paquetes a un receptor más rápido que la capacidad del receptor de procesar cada paquete?** (Asumir que el receptor tiene un búfer para almacenar paquetes que aun no puede procesar)



Bloque 6: Conectar Procesos
Control de Flujo

- **¿Qué pasa si un emisor envía paquetes a un receptor más rápido que la capacidad del receptor de procesar cada paquete?**
  - Se satura de paquetes el búfer del receptor hasta que este ya no puede almacenar más paquetes que le llegan y comienza a perder paquetes.



Bloque 6: Conectar Procesos
Control de Flujo

- **¿Cómo evitar que un emisor rápido sature de paquetes a un receptor lento?** (sugerir ideas)

Bloque 6: Conectar Procesos
Control de Flujo

- **¿Cómo evitar que un emisor rápido sature de paquetes a un receptor lento?**
  - Uso de **retroalimentación al emisor**.
    - O sea, el receptor le indica al emisor cuándo y cuánto puede enviar.

Bloque 6: Conectar Procesos
Comunicación confiable

- El control de flujo evita que el emisor envíe datos más rápido de lo que el receptor puede procesar, protegiendo al host de desbordes y pérdidas locales.
- Pero aun cuando el receptor no se satura, eso no garantiza que la comunicación sea correcta: la red puede perder, duplicar o desordenar paquetes, y los procesos necesitan mecanismos para detectar y corregir esos problemas.
- Por eso, además del control de flujo, es necesario un proceso que asegure una **comunicación confiable** entre procesos, permitiendo reconstruir los datos completos y en el orden adecuado.

Bloque 6: Conectar Procesos
Comunicación confiable

- **Recapitulación**: **¿Como se pueden perder paquetes en una red?**

Bloque 6: Conectar Procesos
Comunicación confiable

- **Recapitulación**: **¿Como se pueden perder paquetes en una red?**
- Los paquetes pueden perderse por varias razones:
  - Por congestión en los enrutadores cuando las colas se llenan;
  - porque el emisor envía más rápido que lo que el receptor puede procesar y este descarta paquetes;
  - por errores físicos o daños en los paquetes durante su transmisión.

Bloque 6: Conectar Procesos
Comunicación confiable

- **¿Como manejar la pérdida de paquetes?** (**ayuda**: a esto ya lo vimos)

Bloque 6: Conectar Procesos
Comunicación confiable

- **¿Como manejar la pérdida de paquetes?** (**ayuda**: a esto ya lo vimos)
- Los paquetes recibidos son confirmados por el receptor.
- El emisor detecta que paquetes faltan.
  - Porque no recibió sus confirmaciones de recepción durante un cierto tiempo.
  - Ese tiempo se mide con un temporizador asociado a cada paquete.
- Si un paquete no es confirmado, el emisor lo retransmite.

Bloque 6: Conectar Procesos
Comunicación confiable

- Estos mecanismos para manejar la pérdida de paquetes permiten recuperar paquetes perdidos, pero no garantizan por sí solos que la comunicación sea completamente correcta:
  - la red puede entregar paquetes duplicados, fuera de orden o dañados.
- Para asegurar que los procesos intercambien datos completos y consistentes, necesitamos mecanismos adicionales que conforman la comunicación confiable entre procesos.
  - Estos los estudiaremos en detalle más adelante.

Bloque 6: Conectar Procesos

- **Recapitulación**:
  - Hasta este punto conectábamos máquinas; ahora conectamos programas dentro de esas máquinas.
  - Entonces los mensajes deben indicar cuál es el programa receptor. Esto introduce los llamados puertos.
  - A su vez aparecen problemas de comunicación entre procesos no resueltos por las capas inferiores como el control de flujo, control de congestión, numeración de mensajes y mecanismos de confiabilidad extremo a extremo.
  - Este bloque nos muestra que la comunicación útil para las aplicaciones requiere abstraerse de los detalles de la red y garantizar orden, integridad y entrega.

Metas

- **Vamos a estudiar**:
  1. Nivel físico
  2. Conexión directa entre máquinas
  3. Conectando redes de área local
  4. Conectando redes de área amplia
  5. Redes proveedoras de contenido
  6. Conectando procesos
  7. **Aplicaciones de red**
  8. Modelo de capas de una red

Bloque 7: Aplicaciones de red

- Aquí pasamos del transporte de datos a la lógica de las aplicaciones.
- Una vez que los procesos pueden comunicarse, podemos construir aplicaciones completas, para esto hay que tomar algunas decisiones de diseño.
- Usamos el ejemplo del pedido de pizza para mostrar cómo se definen tipos de mensajes, estados y reglas de interacción, concluyendo que "a esto se le llama protocolo".
- También introduce arquitecturas como cliente-servidor y peer-to-peer.
- Este bloque responde a: **¿cómo diseñamos aplicaciones que usan la red para ofrecer servicios reales?**

Bloque 7: Aplicaciones de red

- **Ejercicio**: Un proceso Cliente quiere pedir una pizza a un proceso Servidor ubicado en otra máquina. Diseñar cómo se comunican el proceso Cliente y el proceso Servidor para completar un pedido de pizza.
  - ¿Qué datos tiene el pedido del cliente?
  - ¿Cómo sabe el cliente que el servidor recibió el pedido?
  - ¿Cómo representa el servidor el estado del pedido? Supongamos que el pedido va pasando por diferentes estados (recibido, pagado, enviado).
  - ¿Qué datos tienen las respuestas del servidor?

Bloque 7: Aplicaciones de red

- **Tipos de Paquetes**:
  - **Paquete de pedido de pizza** con: tipo y tamaño de pizza, nombre y dirección del cliente, datos de la tarjeta de crédito.
  - **Paquete de confirmación de recepción del pedido** con tipo, tamaño de pizza y nombre del cliente.
  - **Paquete de cobro** con nombre de cliente, tipo y tamaño de pizza y el monto cobrado.
  - **Paquete de envío del pedido** con nombre del cliente, tipo y tamaño de pizza y horario aproximado de entrega y empresa que hace la entrega.
- **Estado del pedido**: tipo y tamaño de pizza, nombre de cliente, y estado (pedido recibido, pedido cobrado, pedido enviado)
- **Reglas de comunicación**: Primero cliente manda paquete de pedido, luego servidor contesta con paquete de confirmación del pedido, luego envía paquete de cobro del pedido y finalmente envía paquete de pedido enviado.

Bloque 7: Aplicaciones de red
Protocolos

- **¿Que pueden observar del ejercicio anterior?**

Bloque 7: Aplicaciones de red
Protocolos

- **¿Que pueden observar del ejercicio anterior?**
  - Hay que definir tipos de mensajes con su estructura
  - Hay que definir reglas de comunicación entre los procesos
  - Hay que definir información de estado
- A esto se le llama **protocolo**.
- Entonces tenemos aplicaciones que proporcionan servicios (llamadas **aplicaciones de red**) definidas mediante protocolos.

Bloque 7: Aplicaciones de red
Protocolos

- Hasta ahora vimos cómo los protocolos de aplicación permiten que dos procesos intercambien mensajes siguiendo reglas bien definidas.
  - Esto es fundamental para diseñar aplicaciones de red.
- Pero antes de escribir un protocolo, necesitamos algo más básico: identificar **qué procesos participan en la aplicación**, **cómo se comunican entre sí** y **qué roles cumple cada uno**.
  - Esa organización previa —quién habla con quién, quién inicia, quién responde, quién almacena, quién distribuye— es lo que se conoce como **arquitectura de la aplicación de red**.
- La arquitectura define la **estructura general de la comunicación**, y el protocolo especifica los detalles de cómo se lleva a cabo.
  - Por eso, antes de diseñar un protocolo de aplicación, necesitamos entender las **distintas arquitecturas posibles**.

Bloque 7: Aplicaciones de red Arquitecturas

- **¿En el ejercicio anterior qué roles cumplen los procesos?**

Bloque 7: Aplicaciones de red
Arquitecturas

- **¿En el ejercicio anterior qué roles cumplen los procesos?**
  - Uno solo pide y el otro solo responde o atiende.
- A esto se le llama **arquitecturas cliente-servidor:** un proceso actúa como cliente (inicia pedidos) y otro como servidor (atiende pedidos).
- Pero no todos las aplicaciones se organizan así.
- En una arquitectura **peer-to-peer**, los procesos pueden cumplir ambos roles: a veces piden, a veces responden, y pueden compartir recursos entre sí sin un servidor central.

Bloque 7: Aplicaciones de red
Arquitecturas

- En general, una **arquitectura** describe qué procesos participan en la aplicación, qué roles cumplen y cómo se comunican entre sí.
- Pensar la arquitectura ayuda a diseñar la aplicación, porque define la estructura básica sobre la cual luego se construyen los protocolos de comunicación.

Bloque 7: Aplicaciones de red

Recapitulación:

- Este bloque nos llevó al nivel donde viven las aplicaciones reales.
- A partir del ejemplo del pedido de pizza, entendimos que una aplicación necesita definir tipos de mensajes, estados y reglas de interacción, conformando lo que se llama un protocolo.
- También exploramos arquitecturas como cliente-servidor y peer-to-peer.
- La idea fundamental es que las aplicaciones no “usan la red” sin más; negocian un protocolo que define cómo cooperan.

Metas

- **Vamos a estudiar**:
  1. Nivel físico
  2. Conexión directa entre máquinas
  3. Conectando redes de área local
  4. Conectando redes de área amplia
  5. Redes proveedoras de contenido
  6. Conectando procesos
  7. Aplicaciones de red
  8. **Modelo de capas de una red**

Bloque 8: Modelo de capas de una red

- Finalmente, integramos todo lo visto antes en un modelo por capas, donde cada nivel ofrece servicios al superior.
- Este bloque responde a: **¿cómo organizamos toda la complejidad de una red en una arquitectura clara, modular y escalable?**

Bloque 8: Modelo de capas de una red

5	Application layer	5: Aplicaciones de red
4	Transport layer	4: Conectar procesos
3	Network layer	3: Conectar LANs 3:Conectar redes
2	Data link layer	2: Conexión directa entre máquinas
1	Physical layer	1:Nivel físico

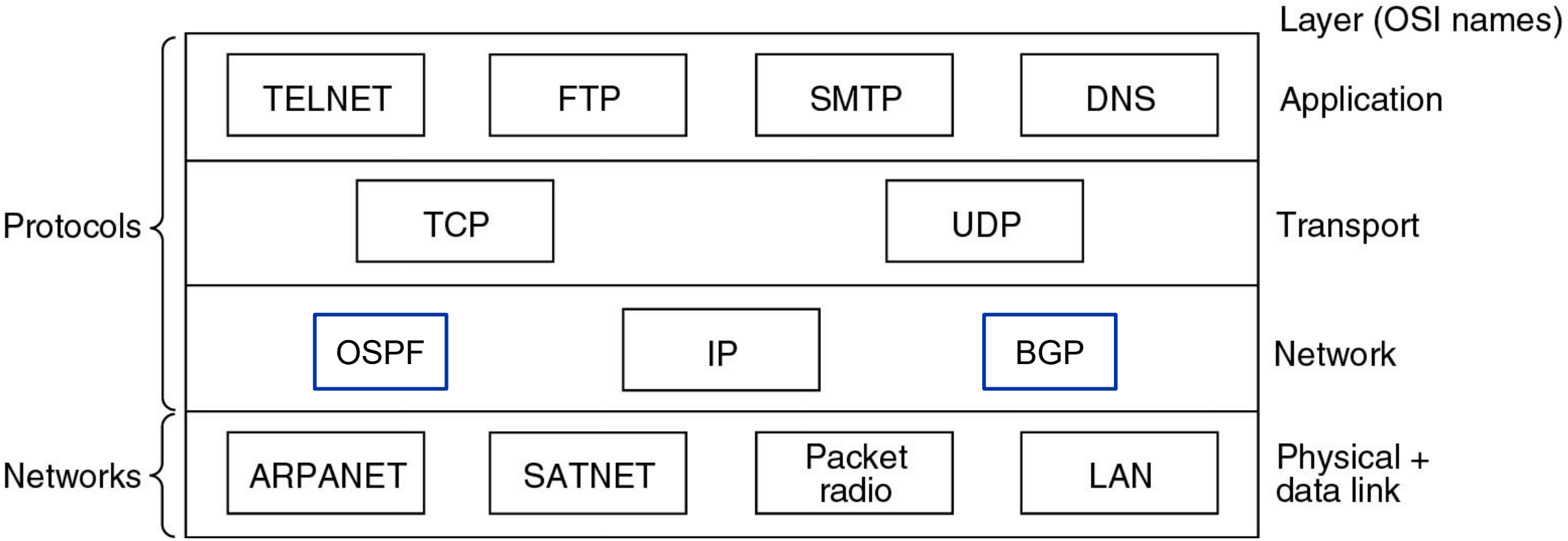
Orden a seguir en la materia: De arriba hacia abajo (Top_down).

Bloque 8: Modelo de capas de una red

Modelo Híbrido

Función		Asuntos/problemas considerados
aplicaciones de red middleware	capa de aplicación	
comunicación entre procesos	capa de transporte	retransmisiones control de flujo control de congestión
envío de paquetes entre 2 hosts usando rutas entre ellos	capa de red	almacenamiento y reenvío enrutamiento control de congestión fragmentación de mensajes
comunicación entre máquinas conectadas directamente entre sí	capa de enlace de datos	control de flujo control de acceso a canal compartido control de errores
transporte usando medios físicos de un stream de datos	capa física	medios físicos guiados y no guiados interconexión de medios físicos de distinto tipo teoría de señales

Bloque 8: Modelo de capas de una red

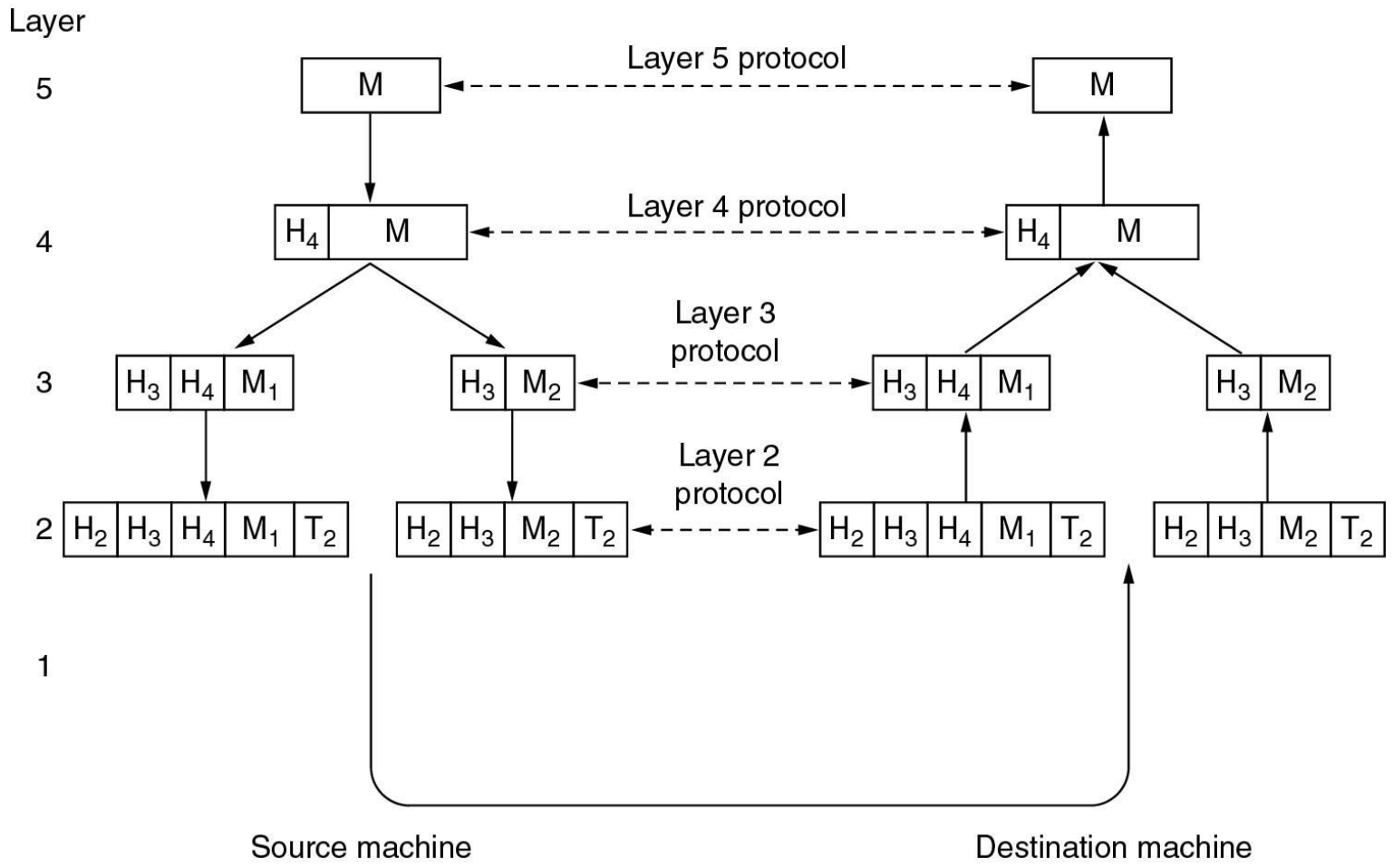


Modelo de referencia TCP/IP

Bloque 8: Modelo de capas de una red

- **Interfaces entre capas** = operaciones y servicios primitivos ofrecidos por una capa a capa superior.
- **Significado de una capa:**
  - Una capa *n* se piensa como una conversación entre la capa *n* de una máquina con la capa *n* de otra máquina;
  - para especificar cómo es esta conversación se definen **protocolos**
- **Protocolo de capa *n*** = reglas y convenciones usadas en la **conversación** entre la capa *n* de una máquina y la capa *n* de otra máquina.
- **Arquitectura de red** = conjunto de capas y protocolos = **pila de protocolos**.

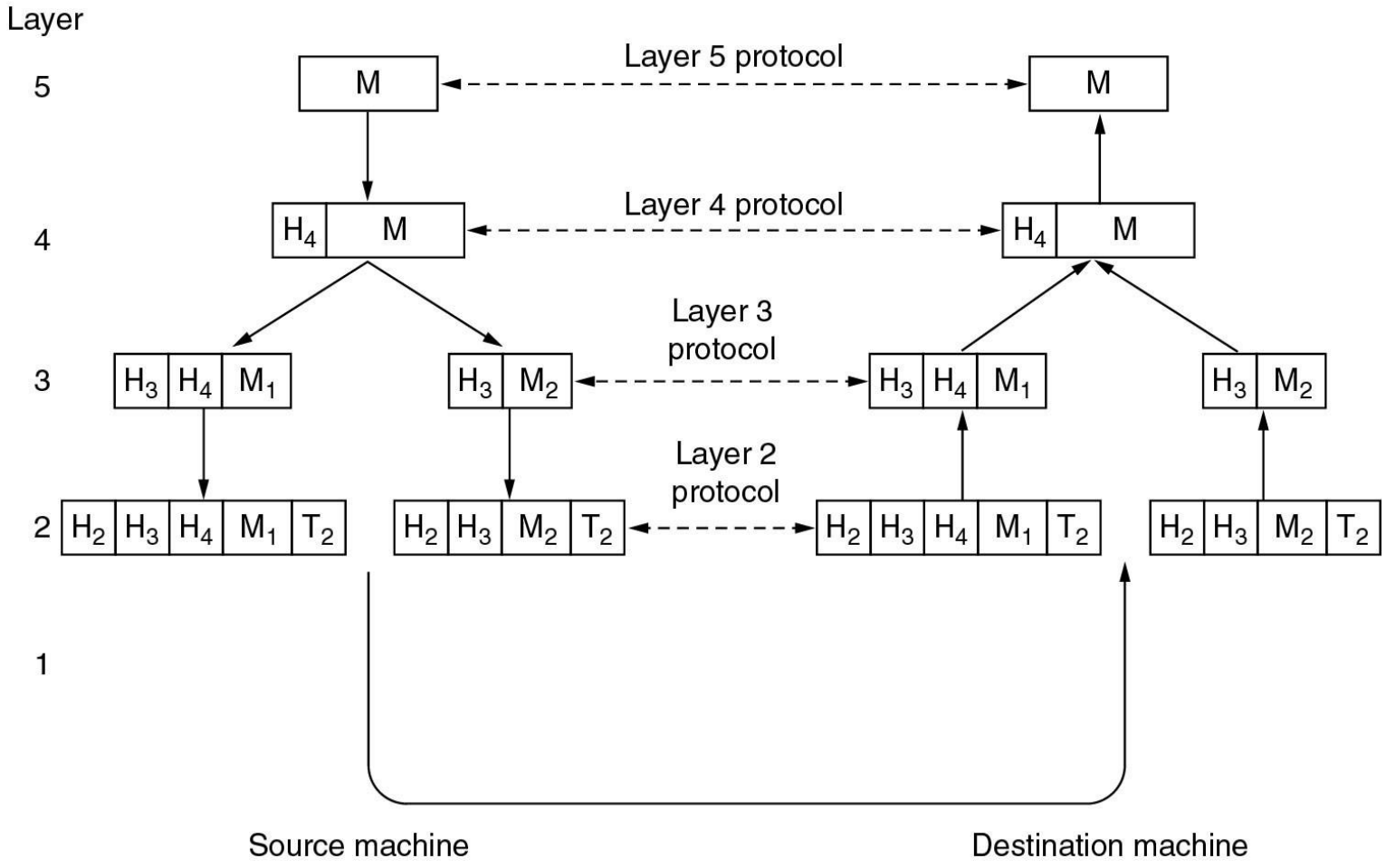
Bloque 8: Modelo de capas de una red



Procesos de aplicación (capa 5 o **capa de aplicación**)

- P.ej. browser, e-mail, chat, ftp, etc.
- Produce un mensaje y lo pasa a la capa 4 para su transmisión.

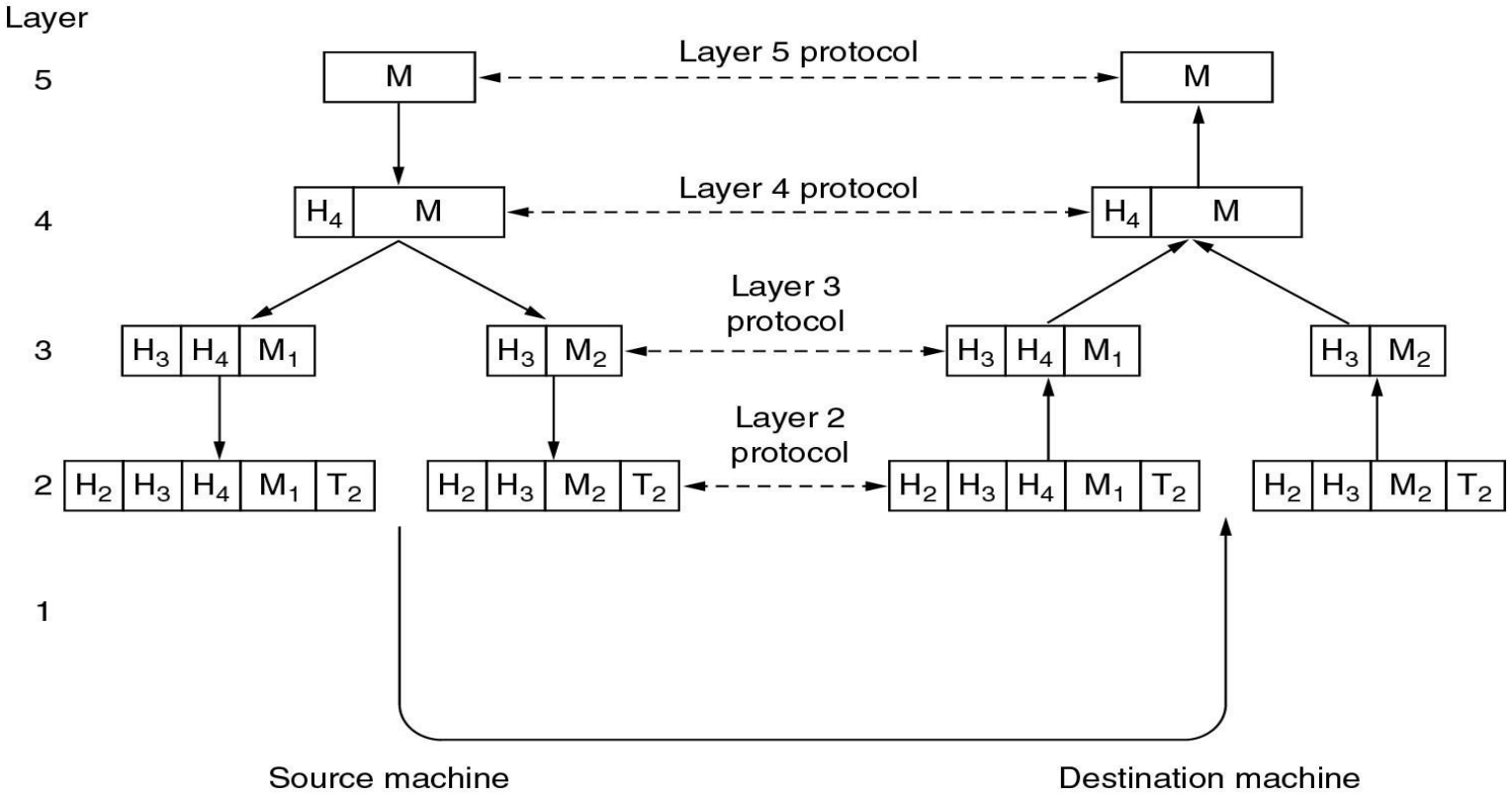
Bloque 8: Modelo de capas de una red



La capa 4 (**capa de transporte**)

- pone un encabezado en el mensaje para identificarlo y pasa el resultado a la capa 3.
- El encabezado contiene **números de secuencia** para que la capa 4 en la máquina de destino entregue los mensajes en el orden correcto.

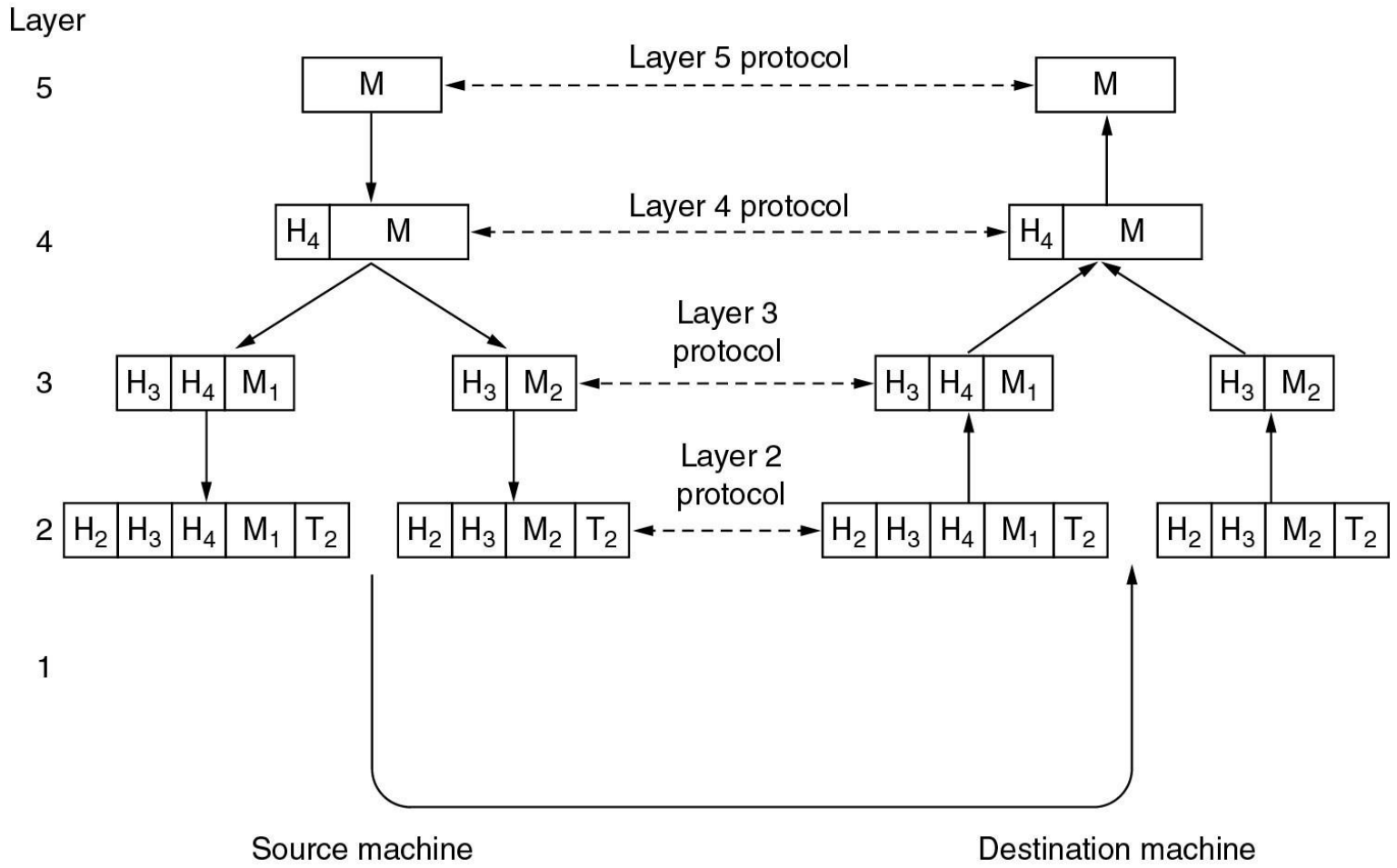
Bloque 8: Modelo de capas de una red



Capa 3 (**capa de red**):

- Hay limitaciones en el tamaño de los mensajes de capa 3.
- Divide en **paquetes** los mensajes que llegan.
- A cada paquete se le coloca un encabezado.
- Decide cuál de las líneas que salen usar (cuando la máquina es un enrutador).
- Pasa los paquetes a la capa 2.

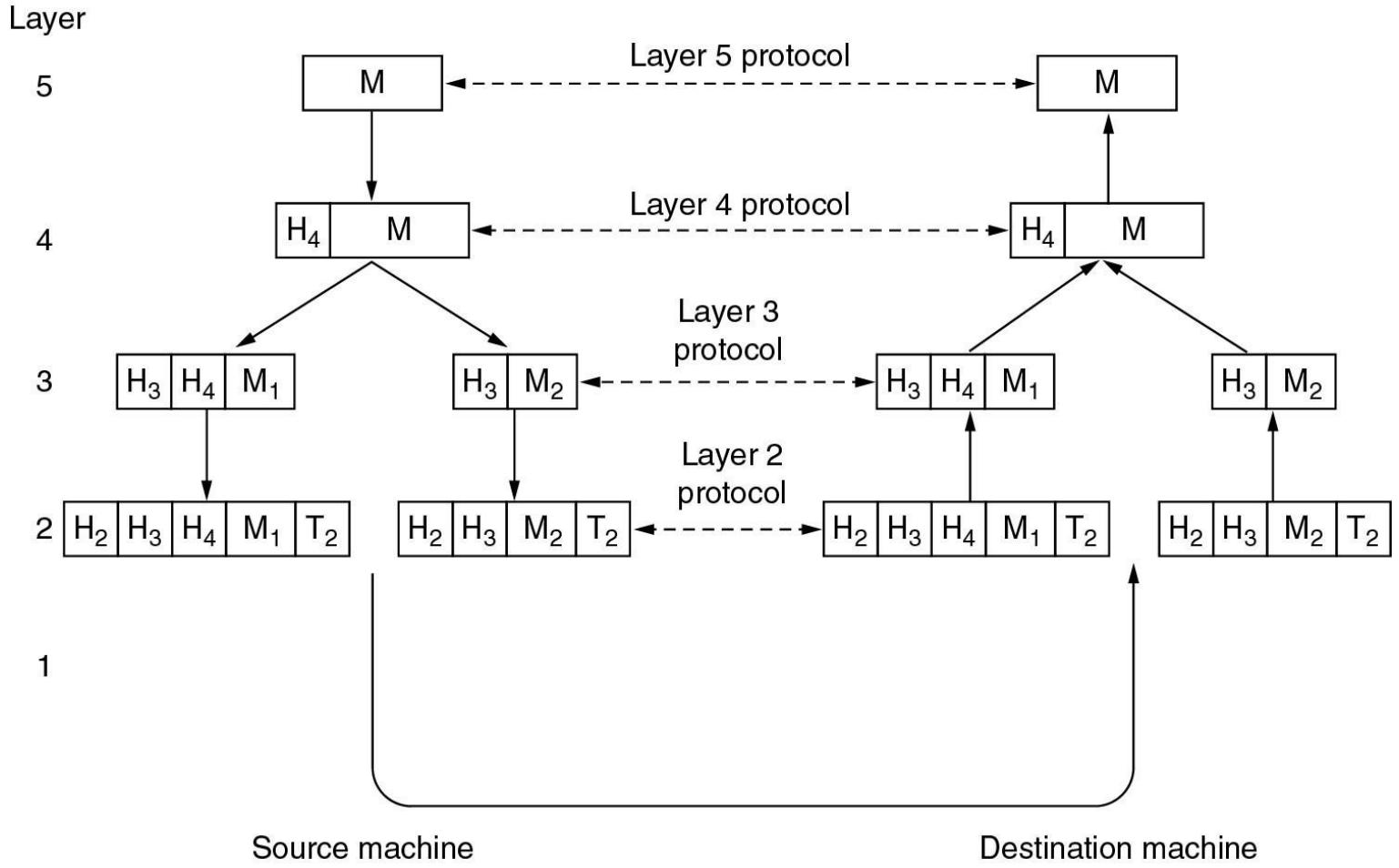
Bloque 8: Modelo de capas de una red



La capa 2 (**capa de enlace de datos**)

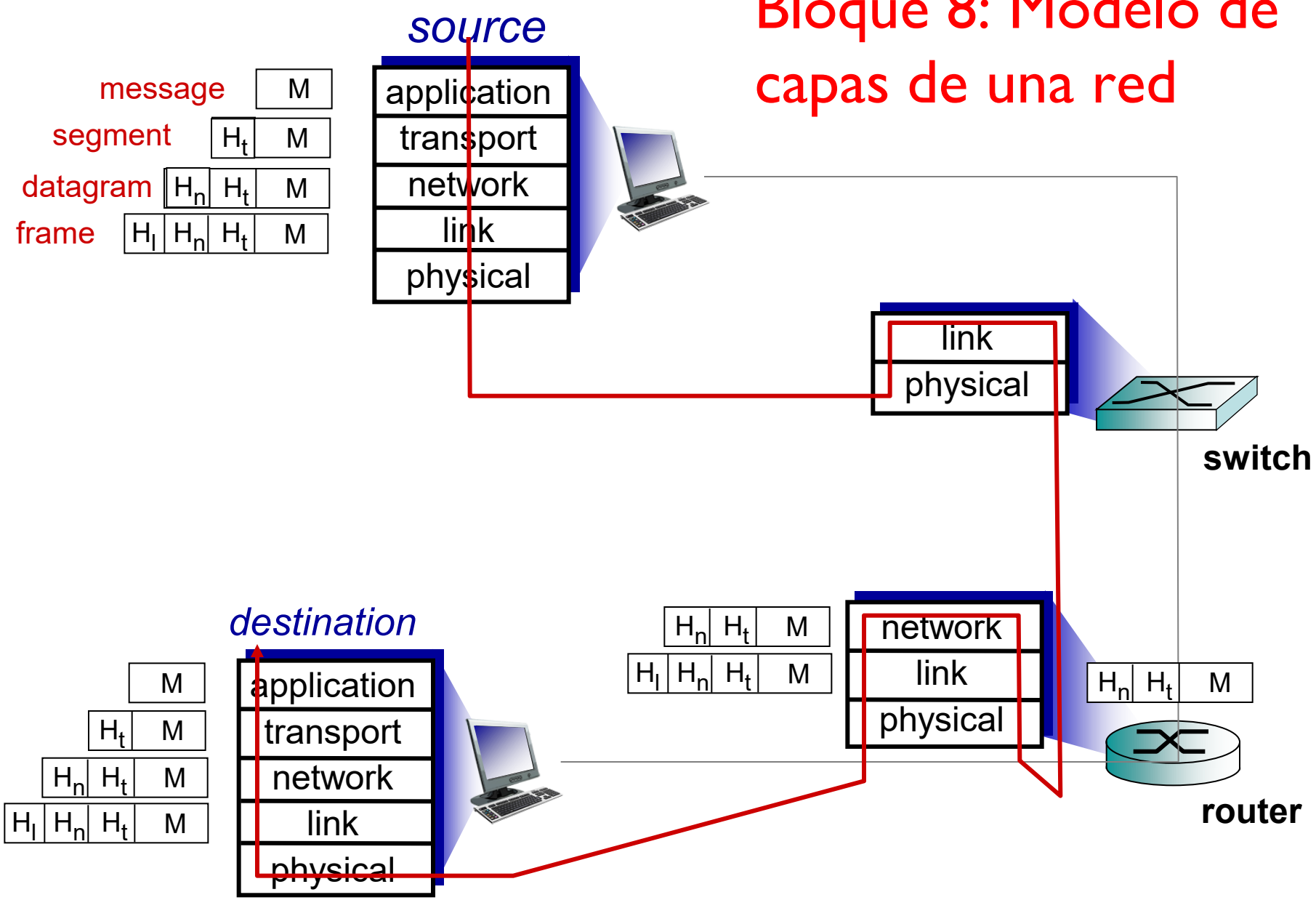
- agrega un encabezado y un terminador, a cada pieza
- pasa la unidad resultante a la capa 1 para su transmisión.

Bloque 8: Modelo de capas de una red



En la máquina receptora el mensaje pasa hacia arriba de capa en capa, perdiendo los encabezados conforme avanza.

Bloque 8: Modelo de capas de una red



Bloque 8: Modelo de capas de una red

Recapitulación:

- Finalmente, integramos todo en un modelo por capas donde cada nivel resuelve un conjunto de problemas y ofrece servicios al siguiente.
- Una capa n se piensa como una conversación entre la capa n de una máquina y la capa n de otra máquina.
- Este bloque nos da la arquitectura conceptual que organiza todo lo aprendido y que sirve para estudiar protocolos concretos como Ethernet, IP, TCP y HTTP.

Cierre del capítulo

- Este capítulo nos permitió recorrer el camino desde la transmisión física de señales hasta la organización global de Internet y la comunicación entre procesos.
- Vimos cómo cada capa resuelve un conjunto específico de problemas y ofrece servicios a la siguiente.
- El resultado es una arquitectura modular que permite que miles de millones de dispositivos y aplicaciones funcionen sin que cada uno deba entender toda la complejidad subyacente.
- Lo que sigue en la materia profundizará en cada una de estas capas, sus protocolos y sus algoritmos.